# 实验三：循环神经网络（RNN）


**实验三：循环神经网络（RNN）**

- 概述
  - 实现内容
- 环境配置
- 任务总览与推荐完成顺序
- Task 1：RNN 单步
  - 步骤 1.1：`rnn_step_forward`
  - 步骤 1.2：`rnn_step_backward`
  - 检查点 A
- Task 2：沿时间展开与 BPTT
  - 步骤 2.1：`rnn_forward`
  - 步骤 2.2：`rnn_backward`
  - 梯度消失与梯度爆炸
  - 步骤 2.3：`clip_grad_norm`
  - 检查点 B
- Task 3：逐时间步的输出层
  - 步骤 3.1：`temporal_affine_forward` / `temporal_affine_backward`
  - 步骤 3.2：`temporal_softmax_loss`
  - 检查点 C
- Task 4：二进制加法器
  - 4.1 任务与数据
    - 步骤 4.1：`add_bits`
    - 步骤 4.2：`make_addition`
  - 4.2 组装 `AdderRNN`
    - 步骤 4.3：`AdderRNN.loss`
    - 步骤 4.4：`AdderRNN.predict`
  - 4.3 训练（`src/train.py`）
    - 步骤 4.5：`sequence_accuracy`
    - 步骤 4.6：`train`
  - 检查点 D：全部通过
  - 4.4 看看它学会了什么（不计分）
- 提供的代码（无需修改）
- 评分标准
  - 提交要求
- 参考资料


## 概述

前两次实验里的 MLP 与 CNN 有一个共同的前提：输入的大小是固定的。但很多数据天然是**序列**（如一句话、一段语音、一条温度曲线）长短不一，而且顺序本身就携带信息。**循环神经网络（Recurrent Neural Network, RNN）**处理序列型数据的方法是维护一个**隐藏状态（hidden state）** $h_t$，逐个读入序列中的元素，每读一个就用**同一组权重**把它更新一次：

$$h_t = \tanh(x_t W_x + h_{t-1} W_h + b)$$

$h_t$ 就是网络的「记忆」：它把到目前为止读过的全部内容压缩进一个定长向量。与 CNN 在**空间**上共享卷积核类似，RNN 在**时间**上共享 $W_x$、$W_h$、$b$，因此参数量与序列长度无关，同一个网络可以读任意长的序列。这一结构可以追溯到 Elman 的工作[1]。

训练 RNN 所用的算法称为**沿时间反向传播（Backpropagation Through Time, BPTT）**[2]：把循环按时间展开成一张普通的计算图，再在这张图上做反向传播。它并不是一个新算法，但带来了 MLP 与 CNN 中都没有的问题：梯度要连续穿过许多个时间步才能传回序列开头，这会导致**梯度消失与梯度爆炸**[3][4]。

本次实验的目标是：**仅使用 NumPy，实现一个 RNN 模型及其 BPTT，并训练该模型，使其学会二进制加法。**该任务之所以必须要用 RNN，是由于 RNN 的「记忆」功能：和的每一位都取决于从低位传上来的进位，而进位只能存在隐藏状态里。训练完成后你会看到，只在 8 位数上训练过的网络能正确计算 128 位数的加法。这说明模型学到的是进位规则本身，并且能够在不同的序列长度上泛化。

## 实现内容

按依赖关系，你需要依次完成以下四个部分：

| 顺序 | 部分 | 内容 | 所在文件 |
|---|---|---|---|
| ① | Task 1 | RNN 单步：`rnn_step_forward` → `rnn_step_backward` | `src/rnn_layers.py` |
| ② | Task 2 | 沿时间展开与 BPTT：`rnn_forward` → `rnn_backward` → `clip_grad_norm` | `src/rnn_layers.py`、`src/optim.py` |
| ③ | Task 3 | 逐时间步的输出层：`temporal_affine_forward` / `temporal_affine_backward` → `temporal_softmax_loss` | `src/temporal_layers.py` |
| ④ | Task 4 | 二进制加法器：`add_bits` → `make_addition` → `AdderRNN.loss` → `AdderRNN.predict` → `sequence_accuracy` → `train` | `src/data.py`、`src/model.py`、`src/train.py` |

> **关于函数签名**：本实验中每个待实现函数的输入输出形状、类型与约定，都已写在 `src/` 下对应函数的类型标注和 docstring 中，实现时请仔细阅读。

---

# 环境配置

请首先安装 `uv` Python 项目管理工具（如果之前的实验已经装过，可跳过）：

```
# On macOS and Linux.
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# On Windows.
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

先确认安装完毕，然后在项目根目录下执行：

```
uv sync
```

该指令会在根目录下创建好 Python 虚拟环境。完成本实验所需的依赖仅存在于该虚拟环境中，若使用 IDE，可先将解释器设置为该环境中的解释器。

之后可以尝试运行测试用例（**路径中不可带非 ascii 编码字符**）：

```
uv run pytest
```

若输出类似以下的内容，则说明环境配置完毕：

```
FFFF.FFFFFFFFFFFFFFFFFFFFF.FFFFFF                                        [100%]
=========================== short test summary info ============================
FAILED tests/test_data.py::test_add_bits_hand_computed - NotImplementedError
FAILED tests/test_data.py::test_add_bits_matches_integer_addition - NotImplem...
FAILED tests/test_data.py::test_make_addition_format_and_correctness - NotImp...
...
33 failed, 2 passed in 0.16s
```

> **关于如何只跑一个测试**：更高效的做法是只跑你正在写的函数的测试，例如：
>
> ```
> uv run pytest tests/test_rnn_step.py                    # 只跑 Task 1 的 4 个测试
> uv run pytest tests/test_rnn_bptt.py -k rnn_forward     # 只跑名字含 rnn_forward 的测试
> uv run pytest tests/test_rnn_bptt.py --tb=short         # 失败时打印简短的报错位置
> ```
>
> 其中 `-k` 后面跟的是测试函数名的**子串**。本文每个步骤末尾都会给出对应的命令。默认配置下 pytest 不打印报错详情，调试时请加上 `--tb=short`。

---

# 任务总览与推荐完成顺序

下面这张表是整个实验的「地图」。**第 3 列「依赖」告诉你必须先完成什么**；建议按从上到下的顺序推进，每完成一行就跑一次对应的验证，再进入下一行。

| 步骤 | 函数/方法 | 依赖 | 完成后跑什么验证 | 所在文件 |
|---|---|---|---|---|
| 1.1 | `rnn_step_forward` | 无 | 2 个 `rnn_step_forward` 测试 | `src/rnn_layers.py` |
| 1.2 | `rnn_step_backward` | 1.1 | `test_rnn_step_backward_gradients` 等 2 个 | `src/rnn_layers.py` |
| 1 | **检查点 A：单步全部通过** | — | `uv run pytest tests/test_rnn_step.py` (4/4) | — |
| 2.1 | `rnn_forward` | 1.1 | 3 个 `rnn_forward` 测试 | `src/rnn_layers.py` |
| 2.2 | `rnn_backward` | 1.2、2.1 | `test_rnn_backward_gradients` 等 3 个 | `src/rnn_layers.py` |
| 2.3 | `clip_grad_norm` | 无 | 2 个 `clip_grad_norm` 测试 | `src/optim.py` |
| 2 | **检查点 B：BPTT 全部通过** | — | `uv run pytest tests/test_rnn_bptt.py` (8/8) | — |
| 3.1 | `temporal_affine_forward` / `temporal_affine_backward` | 无 | 2 个 `temporal_affine` 测试 | `src/temporal_layers.py` |

| 步骤 | 函数/方法 | 依赖 | 完成后跑什么验证 | 所在文件 |
|---|---|---|---|---|
| 3.2 | `temporal_softmax_loss` | 无 | 4 个 `temporal_softmax_loss` 测试 | `src/temporal_layers.py` |
| 3 | **检查点 C：输出层全部通过** | — | `uv run pytest tests/test_temporal.py` (6/6) | — |
| 4.1 | `add_bits` | 无 | 2 个 `add_bits` 测试 | `src/data.py` |
| 4.2 | `make_addition` | 4.1 | 2 个 `make_addition` 测试 | `src/data.py` |
| 4.3 | `AdderRNN.loss` | 检查点 A、B、C，4.2 | `test_gradients_match_numeric` 等 4 个 | `src/model.py` |
| 4.4 | `AdderRNN.predict` | 4.3 | `test_predict_returns_bits` | `src/model.py` |
| 4.5 | `sequence_accuracy` | 4.4 | `test_sequence_accuracy_counts_whole_sequences` | `src/train.py` |
| 4.6 | `train` | 2.3、4.5 | `uv run pytest tests/test_train.py` (7/7) | `src/train.py` |
| 4 | **检查点 D：全部 35 例通过** | — | `uv run pytest` (35/35) | — |

# Task 1：RNN 单步

RNN 的全部计算都由同一个「单步」重复构成：读入当前输入 $x_t$ 与上一步的隐藏状态 $h_{t-1}$，输出新的隐藏状态 $h_t$：

$$h_t = \tanh(a_t), \qquad a_t = x_t W_x + h_{t-1} W_h + b$$

本实验沿用实验二的「行向量」约定：一个 batch 的样本按行堆叠，所以 $x_t$ 的形状为 $(N, D)$，$h_t$ 的形状为 $(N, H)$，$W_x$、$W_h$、$b$ 的形状分别为 $(D, H)$、$(H, H)$、$(H,)$。其中 $N$ 为 batch 大小，$D$ 为每一步输入的维数，$H$ 为隐藏状态的维数。

> **为什么用 tanh**：隐藏状态会被一次次送回网络，有界且以零为中心的 tanh 能防止它随时间步不断放大。无界的激活函数（如 ReLU）用在循环连接上时，隐藏状态更容易在长序列中数值发散。

> 这一整节只允许使用 NumPy，不得调用任何自动微分工具。

## 步骤 1.1：`rnn_step_forward`

**目标**：实现单步前向。计算本身只有一行，但请想清楚 cache 里要存哪些量——下一步的反向传播需要它们。

**函数签名**：

```
def rnn_step_forward(x, prev_h, Wx, Wh, b) -> tuple[Array, Cache]:
    # x: (N, D), prev_h: (N, H), Wx: (D, H), Wh: (H, H), b: (H,)
    # 返回 (next_h: (N, H), cache)
```

**验证**：

```
uv run pytest tests/test_rnn_step.py -k rnn_step_forward
```

应看到 2 passed。

## 步骤 1.2：`rnn_step_backward`

**目标**：单步的反向传播，由 $\partial L/\partial h_t$ 求出对 $x_t$、$h_{t-1}$、$W_x$、$W_h$、$b$ 的梯度。

**原理**：先穿过 tanh。由于 $\tanh'(a) = 1 - \tanh^2(a)$，而 $\tanh(a_t)$ 正是前向已经算出的 $h_t$，因此不必重新计算：

$$\frac{\partial L}{\partial a_t} = \frac{\partial L}{\partial h_t} \odot (1 - h_t^2)$$

其中 $\odot$ 表示逐元素相乘。此后 $a_t = x_t W_x + h_{t-1} W_h + b$ 只是两个仿射变换之和，每一项的反向都与实验二中 `affine_backward` 的推导完全相同。

注意 $1 - h_t^2$ 这一项：当 $|a_t|$ 很大时 tanh 进入饱和区，这一项接近 0，梯度几乎无法穿过。测试 `test_rnn_step_saturated_units_pass_no_gradient` 专门验证这一点——它是 Task 2 中梯度消失的局部原因。

**函数签名**：

```
def rnn_step_backward(dnext_h, cache) -> tuple[Array, Array, Array, Array, Array]:
    # dnext_h: (N, H)，即 ∂L/∂h_t
    # cache: 来自 rnn_step_forward
    # 返回 (dx: (N, D), dprev_h: (N, H), dWx: (D, H), dWh: (H, H), db: (H,))
```

**验证**：

```
uv run pytest tests/test_rnn_step.py -k "rnn_step_backward or saturated"
```

应看到 2 passed。

## 检查点 A

完成以上 2 个函数后，跑：

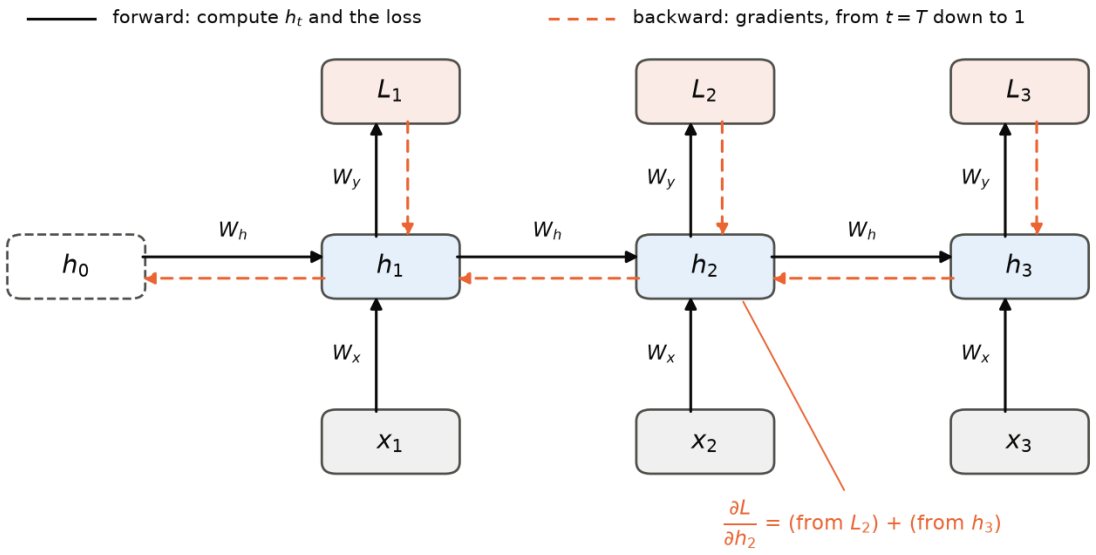
```
uv run pytest tests/test_rnn_step.py
```

应看到 **4 passed**。

---

# Task 2：沿时间展开与 BPTT

把单步沿时间重复 $T$ 次，就得到在整个序列上运行的 RNN。把这 $T$ 步画出来，它就是一张普通的计算图：




The same $W_x$, $W_h$, $b$, $W_y$ are used at every step, so each of their gradients is a sum over $t$.

在这张图上做反向传播，就是 BPTT。与 MLP 相比，有三点需要特别注意：

1. $h_t$ **的梯度有两个来源。**把总损失按时间步拆开，记 $L = L_1 + L_2 + \cdots + L_T$，其中 $L_t$ 是第 $t$ 步的输出所贡献的那部分损失。$L_1, \dots, L_{t-1}$ 与 $h_t$ 无关，于是

   $\frac{\partial L}{\partial h_t} = \frac{\partial L_t}{\partial h_t} + \frac{\partial (L_{t+1} + \cdots + L_T)}{\partial h_t}$

   第一项来自本步的输出层，由调用者通过参数 `dh[:, t]` 直接交给你；第二项只能经由 $h_{t+1}$ 传回来，它恰好就是第 $t+1$ 步反向时返回的 `dprev_h`。所以，反向循环中每一步都要把这两项**加起来**，作为本步 `rnn_step_backward` 的上游梯度。

2. **共享参数的梯度是所有时间步的贡献之和。**同一个 $W_h$ 在每一步都被用到，它的梯度就是每一步各自贡献的累加，例如：

   $\frac{\partial L}{\partial W_h} = \sum_{t=1}^{T} h_{t-1}^{\top} \frac{\partial L}{\partial a_t}$

   这与实验二中卷积核在所有空间位置共享、其梯度要对所有位置求和是同一个道理，只是把「空间」换成了「时间」。

3. **必须从** $t = T$ **倒着算到** $t = 1$。只有 $h_{t+1}$ 的梯度已经完整，才能算出 $h_t$ 的梯度。反向传播要按计算图的逆拓扑序进行，而展开后的 RNN 的逆拓扑序恰好就是时间的倒序。

## 步骤 2.1：`rnn_forward`

**目标**：对 $t = 1, \dots, T$ 依次调用 `rnn_step_forward`，收集每一步的隐藏状态，并保存每一步的 cache 供反向使用。注意返回的 `h` 包含 $h_1, \dots, h_T$，**不包含** $h_0$。

**函数签名**：

```
def rnn_forward(x, h0, Wx, Wh, b) -> tuple[Array, Cache]:
    # x: (N, T, D)，x[:, t] 是第 t 步的输入
    # h0: (N, H)，初始隐藏状态
    # 返回 (h: (N, T, H), cache)
```

**验证**：

```
uv run pytest tests/test_rnn_bptt.py -k rnn_forward
```

应看到 3 passed。其中 `test_rnn_forward_is_causal` 检查的是 RNN 的因果性：$h_t$ 只依赖于 $x_1, \dots, x_t$，改动之后的输入不应影响它。

## 步骤 2.2：`rnn_backward`

**目标**：实现 BPTT。按上面三点，从最后一步开始倒着调用 `rnn_step_backward`，把从下一步传回来的梯度与本步的 `dh[:, t]` 相加，并把每一步的参数梯度累加起来。循环结束时手里的 `dprev_h`，就是对 $h_0$ 的梯度。

**请务必注意参数** `dh` **的含义**：`dh[:, t]` 只是从本步的输出层传到 $h_t$ 的梯度，不包含经由 $h_{t+1}$ 传回来的那部分。

**函数签名**：

```
def rnn_backward(dh, cache) -> tuple[Array, Array, Array, Array, Array]:
    # dh: (N, T, H)，每一步从上方传来的梯度
    # cache: 来自 rnn_forward
    # 返回 (dx: (N, T, D), dh0: (N, H), dWx: (D, H), dWh: (H, H), db: (H,))
```

**验证**：

```
uv run pytest tests/test_rnn_bptt.py -k "rnn_backward or vanishes"
```

应看到 3 passed。

## 梯度消失与梯度爆炸

从 $h_T$ 传回 $h_0$ 的梯度，要依次穿过 $T$ 个单步的反向。由步骤 1.2 可知，每穿过一步，梯度都要先逐元素乘以 $1-h_t^2$（其中每个元素都不超过 1），再乘以 $W_h^\top$。于是经过 $T$ 步之后，梯度的大小大致按 $T$ 次方的规律变化：

- 若每一步都使梯度缩小，它就会随 $T$ **指数衰减**，这就是**梯度消失。**
  `test_gradient_vanishes_over_long_sequences` 用普通的初始化权重做了测量：只在最后一步施加损失时，$\|\partial L/\partial h_0\|$ 在 $T=5$ 时约为 3.1，到 $T=40$ 时只剩约 $2.4\times10^{-8}$，缩小了 8 个数量级。这意味着很久以前的输入几乎得不到任何训练信号，vanilla RNN 因此很难学会跨度达几十步的依赖[3]。LSTM 等门控结构[5]正是为解决这个问题而设计的，有兴趣的同学可以自行学习。
- 反之，若权重使梯度逐步放大，它就可能随 $T$ **指数增长**，这就是**梯度爆炸。**一次过大的梯度就足以让参数一步跳到很远的地方，毁掉之前的训练成果。

梯度爆炸有一个简单有效的对策：**梯度裁剪（gradient clipping）**[4]。把所有参数的梯度看作一个长向量 $g$，若它的范数超过阈值 $\tau$，就把它等比例缩小到 $\tau$：

$$g \leftarrow \begin{cases} g \cdot \dfrac{\tau}{\|g\|}, & \|g\| > \tau \\ g, & \text{否则} \end{cases}$$

## 步骤 2.3：`clip_grad_norm`

**目标**：实现上面的梯度裁剪。注意范数要对**所有参数合在一起**计算（全局范数），而不是对每个参数分别裁剪。

**函数签名**：

```
def clip_grad_norm(grads: dict[str, Array], max_norm: float) -> float:
    # grads: 参数名 -> 梯度，原地修改
    # 返回裁剪之前的全局范数
```

> 本实验最终的加法任务序列很短，即使不裁剪也能训练；但在长序列上，梯度裁剪是训练 RNN 的标准配置，因此 Task 4 的 `train` 默认开启它。PyTorch 中与之对应的是 `torch.nn.utils.clip_grad_norm_`，做的是同一件事。

**验证**：

```
uv run pytest tests/test_rnn_bptt.py -k clip_grad_norm
```

应看到 2 passed。

## 检查点 B

运行：

```
uv run pytest tests/test_rnn_bptt.py
```

应看到 **8 passed**。

---

# Task 3：逐时间步的输出层

二进制加法要求网络在**每一步**都给出一个预测（和的第 $t$ 位）。我们采用一种简单的做法：在每个 $h_t$ 上接**同一个**仿射层得到分数，再在每个位置上各算一次 softmax 交叉熵。这又是一次时间上的权重共享。

## 步骤 3.1：`temporal_affine_forward` / `temporal_affine_backward`

**目标**：把同一个仿射层作用到每个时间步上，并实现其反向。

**原理**：$N$ 个序列、每个 $T$ 步，一共是 $N \times T$ 个互相独立的位置。把形状为 $(N, T, H)$ 的输入看作 $N \times T$ 行的矩阵，问题就退化成了实验二中普通的仿射层；反向时 $W$ 与 $b$ 的梯度要对全部 $N \times T$ 个位置求和。

**函数签名**：

```
def temporal_affine_forward(h, W, b) -> tuple[Array, Cache]:
    # h: (N, T, H), W: (H, M), b: (M,)
    # 返回 (out: (N, T, M), cache)

def temporal_affine_backward(dout, cache) -> tuple[Array, Array, Array]:
    # dout: (N, T, M)
    # 返回 (dh: (N, T, H), dW: (H, M), db: (M,))
```

**验证**：

```
uv run pytest tests/test_temporal.py -k temporal_affine
```

应看到 2 passed。

## 步骤 3.2：`temporal_softmax_loss`

**目标**：在每个位置上计算 softmax 交叉熵，并对全部 $N \times T$ 个位置取平均：

$$L = -\frac{1}{NT}\sum_{i=1}^{N}\sum_{t=1}^{T}\log p_{i,t,y_{i,t}}, \qquad p_{i,t,c} = \frac{e^{s_{i,t,c}}}{\sum_k e^{s_{i,t,k}}}$$

**原理**：取的是**平均**而不是对时间求和，这样损失的大小就不随序列长度 $T$ 变化；每个位置对分数的梯度与实验二的 `softmax_loss` 相同，只是分母由 $N$ 变为 $NT$。

数值稳定性的处理也与实验二相同，并且更严格一些：

- 先减去每个位置分数的最大值，避免 $e^s$ **上溢**为 `inf`；
- 直接计算 $\log p$（即 log-softmax），而不是先算出 $p$ 再取对数。当某一类的分数比最大值低很多时，$e^{s-m}$ 会**下溢**为 0，先算 $p$ 再取对数就会得到 $\log 0 = -\infty$。

> 实际任务（如机器翻译）中，同一个 batch 里的序列长短不一，需要额外传入一个掩码（mask），把填充出来的位置排除在损失之外。本实验中所有序列等长，不需要掩码。

**函数签名**：

```
def temporal_softmax_loss(scores, y) -> tuple[float, Array]:
    # scores: (N, T, C)
    # y: (N, T) 整数标签，取值 [0, C)
    # 返回 (loss: float, dscores: (N, T, C))
```

**验证**：

```
uv run pytest tests/test_temporal.py -k temporal_softmax_loss
```

应看到 4 passed。

## 检查点 C

运行：

```
uv run pytest tests/test_temporal.py
```

应看到 **6 passed**。至此，训练 RNN 所需的全部积木都已就位。

---

# Task 4：二进制加法器

## 4.1 任务与数据

两个二进制数相加时，我们从最低位开始逐位计算：第 $t$ 位的和由 $a_t$、$b_t$ 以及**从低位传上来的进位** $c_t$ 共同决定，同时产生一个新的进位送往高位：

$$s_t = (a_t + b_t + c_t) \bmod 2, \qquad c_{t+1} = \left\lfloor \frac{a_t + b_t + c_t}{2} \right\rfloor, \qquad c_0 = 0$$

也即数字电路中的**全加器（full adder）**。

一个只看当前两个比特的模型（例如在每一位上独立使用的 MLP）无法获取低位的进位信息；而进位只有 1 比特的信息量，正好可以存在 RNN 的隐藏状态里。

数据的格式约定如下：

- **从最低位开始输入。**进位从低位流向高位，所以从最低位开始读，第 $t$ 步所需的信息（低位的进位）都已经读过了。
- **两个加数的最高位固定为 0。**这样和也一定能用同样的 $T$ 位表示，不会溢出。
- 每道题是一个长度为 $T$ 的序列：第 $t$ 步的输入是 $(a_t, b_t)$ 这两个比特，标签是和的第 $t$ 位 $s_t$。

### 步骤 4.1：`add_bits`

**目标**：用一个沿 $t$ 的循环实现上面的全加器链，批量计算 $N$ 组二进制数的和。这正是稍后要让 RNN 学会的算法，请你先亲手实现一遍。

**函数签名**：

```
def add_bits(a, b) -> IntArray:
    # a, b: (N, T) 整数数组，元素为 0/1，最低位在前
    # 返回 (N, T) 整数数组：a + b 的各位，最低位在前（最高位的进位直接丢弃）
```

**验证**：

```
uv run pytest tests/test_data.py -k add_bits
```

应看到 2 passed。

### 步骤 4.2：`make_addition`

**目标**：随机生成一批加法题，并调用 `add_bits` 求出标签。**所有随机数都必须从传入的 `rng` 中抽取**，这样同一个种子总能生成同一批题目。

**函数签名**：

```
def make_addition(num_samples, num_bits, rng) -> tuple[Array, IntArray]:
    # rng: np.random.Generator
    # 返回 (X: (N, T, 2) 浮点数组, y: (N, T) 整数数组)
    # 其中 X[i, t, 0]、X[i, t, 1] 分别是第 i 道题中 a、b 的第 t 位
```

**验证**：

```
uv run pytest tests/test_data.py -k make_addition
```

应看到 2 passed。

## 4.2 组装 `AdderRNN`

请在 `src/model.py` 中补全 `AdderRNN` 类。**构造函数 `__init__` 已经写好，你只需要补全 `loss` 与 `predict`**。网络结构如下，每个时间步使用同一组参数：

```
x_t = (a_t, b_t)  →  h_t = tanh(x_t Wx + h_{t-1} Wh + b)  →  scores_t = h_t Wy + by  →
softmax 交叉熵（标签 s_t）
    (2 维)                        (H 维)                          (2 维：第 t 位是 0 / 是 1)
```

- 初始隐藏状态 $h_0$ 取全零。
- 参数以 `dict` 存储（`Wx / Wh / b / Wy / by`），梯度同样以 `dict` 返回，这样 `optim.py` 中的 `adam` 可以直接逐参数套用。
- 权重用 $\mathcal{N}(0, 1/\text{fan_in})$ 初始化（这是 tanh 网络常用的 LeCun 初始化），偏置初始化为零。
- 默认 $H = 16$ 时，整个网络只有 **338 个参数**，而且其中没有任何一个的形状与序列长度 $T$ 有关。

### 步骤 4.3：`AdderRNN.loss`

**目标**：前向依次调用 `rnn_forward` → `temporal_affine_forward` → `temporal_softmax_loss`；反向按相反的顺序调用 `temporal_affine_backward` → `rnn_backward`，把各参数的梯度填进 `grads`。

**函数签名**：

```
def loss(self, X, y=None):
    # X: (N, T, 2)；y: (N, T) 整数标签
    # y=None → 返回 scores: (N, T, 2)
    # 否则   → 返回 (loss, grads)，grads 的键与 self.params 相同
```

其中 `test_initial_loss_is_about_log2` 是一个健全性检查：未训练的网络对每一位都近似于瞎猜，损失应当在 $\ln 2 \approx 0.693$ 附近。若差得很远，先检查损失是否正确地对 $N \times T$ 个位置取了平均。

**验证**：

```
uv run pytest tests/test_model.py -k "scores_shape or any_length or initial_loss or
gradients_match"
```

应看到 4 passed。

### 步骤 4.4：`AdderRNN.predict`

**目标**：在每个位置上取分数较大的那一类，作为和的这一位。

**函数签名**：

```
def predict(self, X) -> IntArray:
    # X: (N, T, 2)
    # 返回 (N, T) 整数数组，元素为 0/1
```

**验证**：

```
uv run pytest tests/test_model.py -k predict
```

应看到 1 passed。

## 4.3 训练（`src/train.py`）

`TrainConfig` dataclass 已给出，默认值为：隐藏层 $H = 16$，每道题 8 位，batch 大小 64，训练 1000 步，Adam 学习率 $10^{-2}$，梯度裁剪阈值 5.0。

与实验二不同，这里没有固定的训练集：**每一步都现场生成一批新题目**，模型从不会两次见到同一道题，因此也无所谓过拟合，训练按步数而不是按 epoch 计。

### 步骤 4.5：`sequence_accuracy`

**目标**：统计**整道题完全答对**的比例（$T$ 位全部正确才算对）。

**原理**：对加法来说，8 位里错 1 位就是错误答案（事实证明，逐位准确率会严重高估模型的能力）。

**函数签名**：

```
def sequence_accuracy(net, X, y) -> float:
    # X: (N, T, 2), y: (N, T)
    # 返回 [0, 1] 之间的 float
```

**验证**：

```
uv run pytest tests/test_train.py -k sequence_accuracy
```

应看到 1 passed。

### 步骤 4.6：`train`

**函数签名**：

```
def train(cfg: TrainConfig, verbose=False) -> tuple[AdderRNN, dict[str, list[float]]]:
    # 返回 (net, history)
```

**实现要求**：

1. 用 `np.random.default_rng(cfg.seed)` 创建**一个**随机数生成器，之后所有批次都从它抽取；构造 `AdderRNN(cfg.hidden_dim, seed=cfg.seed)`，并为每个参数准备一个优化器 config（`{"learning_rate": cfg.lr}`）。
2. 若 `cfg.recurrent` 为 `False`，在训练开始前把 `Wh` 置零，并且整个训练过程中都不更新它。
3. 每一步：用 `make_addition` 生成一批题目 → 调用 `net.loss` 得到损失与梯度 → 调用 `clip_grad_norm`（`cfg.clip` 为 `None` 时不裁剪，但仍要测量范数，可以传入 `np.inf`）→ 用 `adam` 更新每个参数。
4. 把每一步**更新之前**的损失、**裁剪之前**的梯度范数分别记入 `history["loss"]` 与 `history["grad_norm"]`。
5. `verbose=True` 时每 100 步打印一次进度。

`recurrent=False` 是一个对照实验：切断时间步之间的连接之后，每一步只能看到自己的两个输入比特，网络就退化成了一个在每一位上独立使用的 MLP。`test_without_recurrence_it_cannot_add` 验证它确实学不会加法。

**验证**：

```
uv run pytest tests/test_train.py
```

应看到 **7 passed**。其中 `test_rnn_learns_binary_addition` 要求训练后 8 位加法的整题准确率超过 99%，`test_rnn_generalizes_to_longer_numbers` 则用**同一个**只在 8 位数上训练过的网络去做 32 位加法，同样要求超过 99%。

## 检查点 D：全部通过

```
uv run pytest
```

应看到 **35 passed**。至此，你的 RNN 实现已经通过了所有自动化验证。

---

## 4.4 看看它学会了什么（不计分）

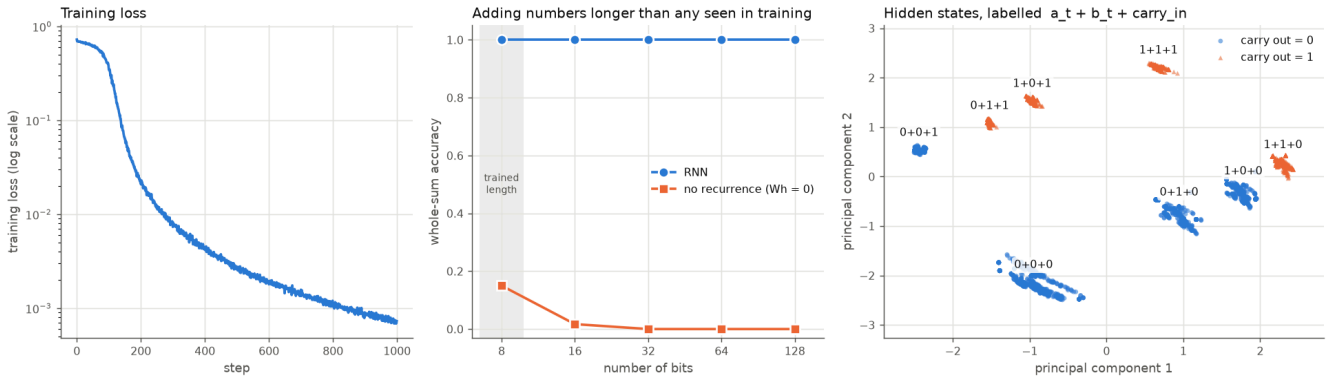
全部测试通过后，运行：

```
uv run python demo.py
```

脚本会用默认配置训练网络（约 1 秒），再训练一个 `recurrent=False` 的对照组，然后在比训练时长得多的数上测试二者：

```
whole-sum accuracy on 1000 random problems (trained on 8-bit numbers only)
  bits      RNN   no recurrence
     8    1.000           0.150
    16    1.000           0.017
    32    1.000           0.000
    64    1.000           0.000
   128    1.000           0.000
```

并把下面这张图保存到 `outputs/rnn_adder.png`：




- **左**：训练损失（对数坐标）。
- **中**：只在 8 位数上训练过的 RNN，在 128 位的加法上依然全部答对。去掉循环连接的对照组的能力范围，就是逐位计算 $a_t \oplus b_t$ 并假设永远没有进位，因此只有当整道题恰好不产生任何进位时才答得对；而在 8 位的题目里，这样的题约占 $(3/4)^7 \approx 13\%$；位数越多，这个比例越趋近于 0。
- **右**：把所有时间步的隐藏状态 $h_t$ 投影到它们的前两个主成分上。点恰好聚成 **8 团**，每一团旁边标注的是该步的 $a_t + b_t + c_t$：正好是全加器的全部 $2^3 = 8$ 种输入。颜色表示这一步产生的**向上的进位**：有进位的 4 团与没有进位的 4 团，恰好对应 $a_t + b_t + c_t \geq 2$ 与 $< 2$。网络在隐藏状态里自己建立了一张全加器的真值表，而**从头到尾，没有人告诉过它「进位」是什么。**

你还可以让训练好的网络去算任意大的数：

```
uv run python demo.py 123456789012345678901234567890 98765432109876543210987654321
```

```
the RNN says 123456789012345678901234567890 + 98765432109876543210987654321 =
111111111011111111101111111100   (correct)
```

# 提供的代码（无需修改）

以下内容已经写好，直接使用：

| 文件 | 内容 |
| --- | --- |
| `src/optim.py` 中的 `adam` | Adam 优化器（与实验二相同） |
| `src/model.py` 中的 `AdderRNN.__init__` | 参数的创建与初始化 |
| `src/train.py` 中的 `TrainConfig` | 训练超参数 |

| 文件 | 内容 |
| --- | --- |
| `src/data.py` 中的 `int_to_bits` / `bits_to_int` | Python 整数与比特向量互相转换（供 `demo.py` 使用） |
| `tests/gradient_check.py` | 数值梯度检验工具 |
| `demo.py` | 训练、长度泛化测试、画图，以及用训练好的网络做加法（不计分） |

# 评分标准

本次实验满分 **100 分**，全部由自动化测试决定。`uv run pytest` 共 35 例，得分按通过比例计算：

$$\text{得分} = \frac{\text{通过用例数}}{35} \times 100$$

**请假定助教不会逐行阅读你的源代码，仅通过运行测试用例计算分数。**但**会对代码进行抽查**，不得针对测试用例硬编码、或以其他方式绕过计算。

## 提交要求

请按要求完成实验代码，之后：

1. 将根目录下的 `.venv`、`.pytest_cache`、`outputs` 目录删除（如有）；
2. 将根目录（`lab3`）以 zip 格式打包提交，压缩包命名为 `学号-姓名-lab3`。

注意：实验过程中，除了 `uv` 自动生成的文件，你只能修改 `src` 下的源文件（`demo.py` 不计分，可随意修改）；`tests` 目录下的任何文件都不得修改；

提交 ddl：2026 年 10 月 28 日。

# 参考资料


[1] Elman, J. L. (1990). Finding Structure in Time. *Cognitive Science*, 14(2), 179–211. https://doi.org/10.1207/s15516709cog1402_1 提出了本实验所用的「隐藏状态反馈给自身」的简单循环网络（常称 Elman 网络）。

[2] Werbos, P. J. (1990). Backpropagation Through Time: What It Does and How to Do It. *Proceedings of the IEEE*, 78(10), 1550–1560. https://doi.org/10.1109/5.58337 系统阐述了 BPTT：把循环按时间展开后做普通的反向传播。

[3] Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning Long-Term Dependencies with Gradient Descent is Difficult. *IEEE Transactions on Neural Networks*, 5(2), 157–166. https://doi.org/10.1109/72.279181 最早系统分析梯度消失问题的工作之一，解释了为什么 vanilla RNN 难以学会长距离依赖。

[4] Pascanu, R., Mikolov, T., & Bengio, Y. (2013). On the Difficulty of Training Recurrent Neural Networks. *ICML*. https://arxiv.org/abs/1211.5063 从动力系统的角度分析梯度消失与爆炸，并提出了按全局范数裁剪梯度的方法。

[5] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735 LSTM 的原始论文，通过门控结构让梯度可以跨越很长的时间跨度而不消失，是 vanilla RNN 之后最重要的循环结构。